

LAB 5 SUBMISSION

C027-01-2727/2024 MARY SHARNICE OTIENO

CODES FOR ALL THE ENDPOINT

CREATE BOOK

```
@app.post("/books", response_model=Book, status_code=201)
def create_book(book: BookCreate, session: Session = Depends(get_session)):
    """Create a new book"""

    existing = session.exec(
        select(Book).where(Book.isbn == book.isbn)
    ).first()

    if existing:
        raise HTTPException(
            status_code=400,
            detail="ISBN already exists"
        )

    db_book = Book(**book.model_dump())

    session.add(db_book)
    session.commit()
```

```
session.refresh(db_book)
```

```
return db_book
```

LIST BOOKS

```
@app.get("/books", response_model=List[Book])
```

```
def list_books(
```

```
    skip: int = 0,
```

```
    limit: int = 10,
```

```
    author: Optional[str] = None,
```

```
    min_price: Optional[float] = None,
```

```
    max_price: Optional[float] = None,
```

```
    available: Optional[bool] = None,
```

```
    session: Session = Depends(get_session)
```

```
):
```

```
    """List books with optional filters"""
```

```
    query = select(Book)
```

```
    if author:
```

```
        query = query.where(Book.author == author)
```

if min_price is not None:

 query = query.where(Book.price >= min_price)

if max_price is not None:

 query = query.where(Book.price <= max_price)

if available is not None:

 query = query.where(Book.available == available)

return session.exec(

 query.offset(skip).limit(limit)

).all()

GET ONE BOOK

@app.get("/books/{book_id}", response_model=Book)

def get_book(

 book_id: int,

 session: Session = Depends(get_session)

):

 """Get a book by ID"""

 book = session.get(Book, book_id)

```
if not book:
    raise HTTPException(
        status_code=404,
        detail="Book not found"
    )
```

```
return book
```

UPDATE BOOK

```
@app.patch("/books/{book_id}", response_model=Book)
def update_book(
    book_id: int,
    book_update: BookUpdate,
    session: Session = Depends(get_session)
):
    """Update a book"""

    book = session.get(Book, book_id)

    if not book:
```

```
raise HTTPException(  
    status_code=404,  
    detail="Book not found"  
)
```

```
updates = book_update.model_dump(exclude_unset=True)
```

```
for key, value in updates.items():  
    setattr(book, key, value)
```

```
book.updated_at = datetime.utcnow()
```

```
session.commit()  
session.refresh(book)
```

```
return book
```

DELETE BOOK

```
@app.delete("/books/{book_id}", status_code=204)  
def delete_book(  
    book_id: int,  
    session: Session = Depends(get_session)
```

```
):  
    """Delete a book"""  
  
    book = session.get(Book, book_id)
```

```
    if not book:  
        raise HTTPException(  
            status_code=404,  
            detail="Book not found"  
        )
```

```
    session.delete(book)  
    session.commit()
```

```
    return None
```

SEARCH BOOKS

```
@app.get("/books/search", response_model=List[Book])  
def search_books(  
    q: str,  
    session: Session = Depends(get_session)  
):  
    """Search books by title or author"""
```

```
query = select(Book).where(
    (Book.title.contains(q))
    |
    (Book.author.contains(q))
)

return session.exec(query).all()
```

SCREENSHOT FOR THE SWAGGER UI

MY Bookstore API C027-01-2727/2024

/openapi.json

default			^
POST	/books	Create Book	▼
GET	/books	List Books	▼
GET	/books/{book_id}	Get Book	☑
PATCH	/books/{book_id}	Update Book	▼
DELETE	/books/{book_id}	Delete Book	▼
GET	/books/search	Search Books	▼
Schemas			^
Book > Expand all object			
BookCreate > Expand all object			